# DocCrate Studio

System Architecture & Technical Specification

**Version 1.0.1 Release**

Prepared internally for architectural review and intellectual property documentation.

# 1. Executive Summary

DocCrate Studio is a natively compiled, cross-platform desktop application designed to solve the problem of offline knowledge management. It provides a secure, air-gapped environment to compile public Git repositories and Markdown documents into highly compressed, searchable, proprietary archives (`.docpack`).

The application is built using the Tauri framework, leveraging a strictly typed Rust backend for high-performance file operations and memory safety, paired with a lightweight web-based frontend for a responsive user interface. This document outlines the structural pillars of the application and the data flow between them.

# 2. The 3-Tier Architecture

DocCrate Studio enforces a strict separation of concerns through a 3-tier architecture. This design guarantees that the presentation layer is entirely sandboxed and cannot directly interact with the host operating system, preventing malicious payload execution.

### Tier 1: The Presentation Layer (Frontend)

**Location:** `ui/index.html`

**Stack:** HTML5, CSS3 (Flexbox), Vanilla JavaScript.

**Responsibility:** This tier is the "face" of the application. It handles all user interactions, tab routing (Publisher vs. Reader modes), and responsive layout rendering. It is intentionally "dumb" — it possesses no native capabilities. It relies entirely on Inter-Process Communication (IPC) to request data or actions from the underlying host.

### Tier 2: The Application Bridge (Tauri)

**Location:** `crates/desktop/src/main.rs`

**Stack:** Rust, Tauri.

**Responsibility:** Acting as the "nervous system," the Bridge initializes the OS-level webview window and establishes the secure IPC boundary. It registers exposed asynchronous commands (using the `#[command]` macro) and securely passes messages from the Frontend to the Core Engine. It utilizes the `tokio` runtime to spawn blocking tasks onto background threads, ensuring the UI remains perfectly fluid during heavy I/O operations.

### Tier 3: The Core Engine (Backend Muscle)

**Location:** `crates/core/src/lib.rs`

**Stack:** Pure Rust (`walkdir`, `zip`, `pulldown-cmark`).

**Responsibility:** This tier performs all the heavy lifting. It executes system-native `git clone` commands, parses directories, compresses data into `.docpack` zip archives, natively translates raw Markdown into styled HTML, and performs high-speed linear searches across packed text files. Because this tier is isolated in its own crate, the core logic is entirely decoupled from the UI framework.

# 3. Execution Lifecycle & Data Flow

To understand how the system operates under load, the following maps the exact execution lifecycle when a user initiates the creation of a new archive.

## The "Build & Pack" Sequence

**1. Intent Capture:** The user clicks "Build & Pack Library" in the UI. The JavaScript disables the button to prevent duplicate triggers and invokes the IPC command via `window.__TAURI__.tauri.invoke('build_library')`.

**2. Thread Delegation:** The Tauri Bridge receives the command. Recognizing that I/O operations (cloning, zipping) will block the main thread, it immediately pushes the task to a background thread using `tokio::task::spawn_blocking`.

**3. Engine Initialization:** The Core Engine initializes a temporary staging directory (`doccrate_staging`) and reads the staged Git URLs.

**4. Native Subprocessing:** The Engine spawns child processes calling the host machine's native `git` command. It forces shallow clones (`--depth 1`) to drastically reduce bandwidth and disk usage.

**5. Compression & Assembly:** Using `walkdir`, the Engine recursively scans the staged directories, filtering exclusively for `.md` files. It opens a `ZipWriter` and streams the file buffers directly into `Master_Library_v1.docpack`.

**6. Garbage Collection & Return:** The Engine deletes the temporary staging directory to prevent memory leaks and returns a success payload to the Bridge, which resolves the JavaScript Promise in the UI.

# 4. Testing and Verification

The Core Engine includes a comprehensive, automated test suite embedded within the Rust binary. During the GitHub Actions CI/CD pipeline, the suite programmatically generates virtual `.docpack` files, populates them with dummy text, and mathematically verifies that the reader, the HTML parser, and the global search engine function flawlessly prior to compilation. This ensures total logic integrity for every release.